

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

*I __S.DIVYASRI(192524388)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: S.DIVYASRI

Design Task

1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration

Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

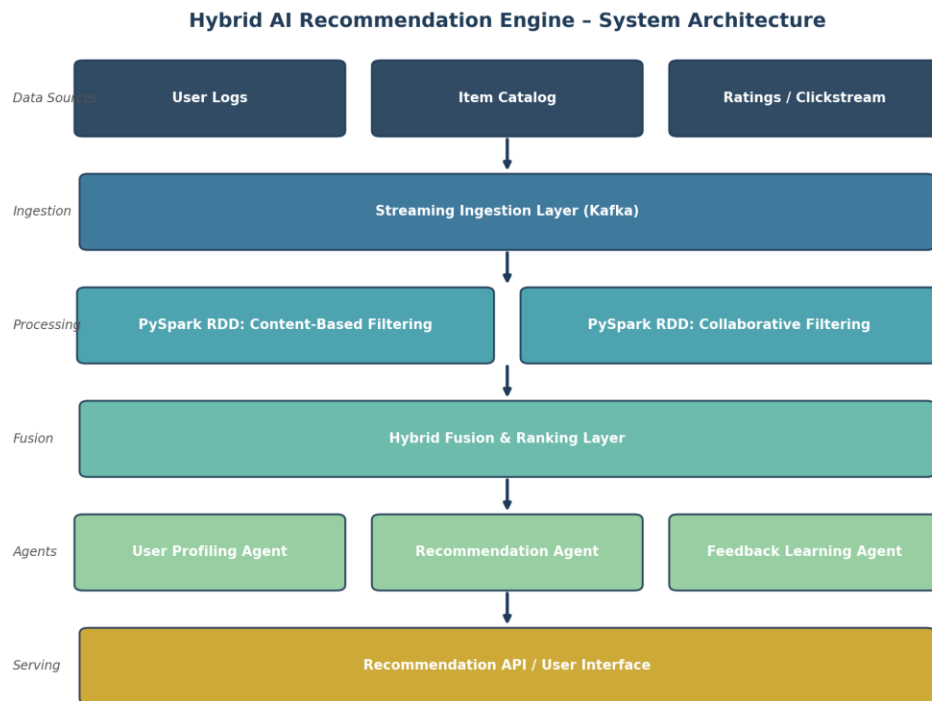
Question 1: Hybrid AI Recommendation Engine Design

1. Problem Understanding & Requirement Analysis

The goal is to design a recommendation engine that overcomes the individual weaknesses of content-based and collaborative filtering. Content-based filtering alone leads to over-specialised suggestions and cannot recommend items outside a user's known interests, whereas collaborative filtering alone suffers from the cold-start problem for new users and new items. The system therefore needs to (i) ingest and process very large volumes of continuously growing user-item interaction data, (ii) fuse content-based and collaborative similarity scores into a single ranked recommendation list, (iii) adapt in near real time as user behaviour changes, and (iv) scale horizontally as the user base and item catalogue grow. Key constraints include low-latency response for the end user, fault tolerance in the processing pipeline, and the ability to incorporate both implicit feedback (clicks, dwell time, add-to-cart events) and explicit feedback (ratings, likes).

2. Proposed System Architecture

The proposed architecture is organised into six layers. The data-source layer collects user logs, the item catalogue, and ratings or clickstream events. A streaming ingestion layer buffers this data and forwards it into the processing layer. The processing layer runs two parallel PySpark RDD pipelines — one performing content-based similarity computation on item feature vectors, and another performing collaborative filtering on the user-item interaction matrix. A hybrid fusion layer combines the two sets of scores using an adaptive weighting scheme. An intelligent-agent layer then personalises and re-ranks the fused results before they reach the serving layer, which exposes the final recommendations through an API or user interface.



3. PySpark & RDD-Based Distributed Processing Design

Raw interaction logs are transformed into structured (user, item, rating/event) tuples using map and flatMap transformations, and filter is used to discard malformed or duplicate records. For the content-based branch, item-feature RDDs are built from catalogue metadata and item similarity is computed using cosine similarity, with a broadcast variable used to share the compact item-feature matrix to every executor so that no expensive shuffle is required. For the collaborative branch, the user-item RDD is partitioned by user-id hash and processed with an alternating-least-squares-style latent-factor computation, where reduceByKey aggregates partial gradients across partitions and mapPartitions is used to batch updates efficiently within each executor. Frequently reused RDDs, such as the cleaned interaction RDD, are persisted in memory to avoid recomputation across iterations, and accumulators track the number of interactions processed for monitoring purposes. Spark's lazy evaluation and DAG-based execution plan ensure that transformations are optimised and executed only when an action (such as collect or saveAsTextFile) is triggered.

4. Intelligent Agent Integration

Three cooperating agents drive personalisation. The User Profiling Agent continuously updates each user's embedding vector from the latest RDD outputs, capturing evolving interests. The Recommendation Agent merges the content-based and collaborative scores, applying contextual re-ranking based on signals such

as time of day or session context. The Feedback Learning Agent monitors implicit and explicit feedback as it arrives and adjusts the fusion weight between the content-based and collaborative components — for example, increasing the collaborative weight once a user has accumulated sufficient interaction history. The agents communicate asynchronously through a lightweight message bus, allowing the recommendation logic to adapt incrementally without requiring a full batch retrain of the underlying models.

5. Scalability & Distributed Computing Design

Horizontal scalability is achieved by partitioning RDDs on user-id hash so that executor load is evenly distributed, and by relying on Spark's RDD lineage for automatic fault recovery — if an executor fails, only the lost partitions are recomputed from lineage rather than the entire dataset. For the iterative latent-factor computation, periodic checkpointing truncates the lineage graph to prevent excessive recomputation overhead across iterations. Structured streaming micro-batches allow the ingestion layer to process near-real-time data without the operational complexity of a fully event-driven system, and Spark's dynamic resource allocation scales the number of executors up or down according to workload.

6. Data Flow & Processing Pipeline

The pipeline flows as follows: raw interaction logs → streaming ingestion → cleaning and parsing RDDs → parallel content-based and collaborative RDD processing → hybrid fusion and ranking → agent-based contextual re-ranking → caching of top-N recommendations in a low-latency store → delivery through the recommendation API. Each stage consumes the RDD produced by the previous stage, preserving a clear, auditable, and independently scalable data flow.

7. Innovation & Novel Aspects

The design's novelty lies in the adaptive (rather than static) weighting between content-based and collaborative components, driven directly by an intelligent agent that reacts to real-time feedback, and in the use of contextual re-ranking to make recommendations sensitive to session-level signals — distinguishing it from conventional batch-only hybrid recommenders.

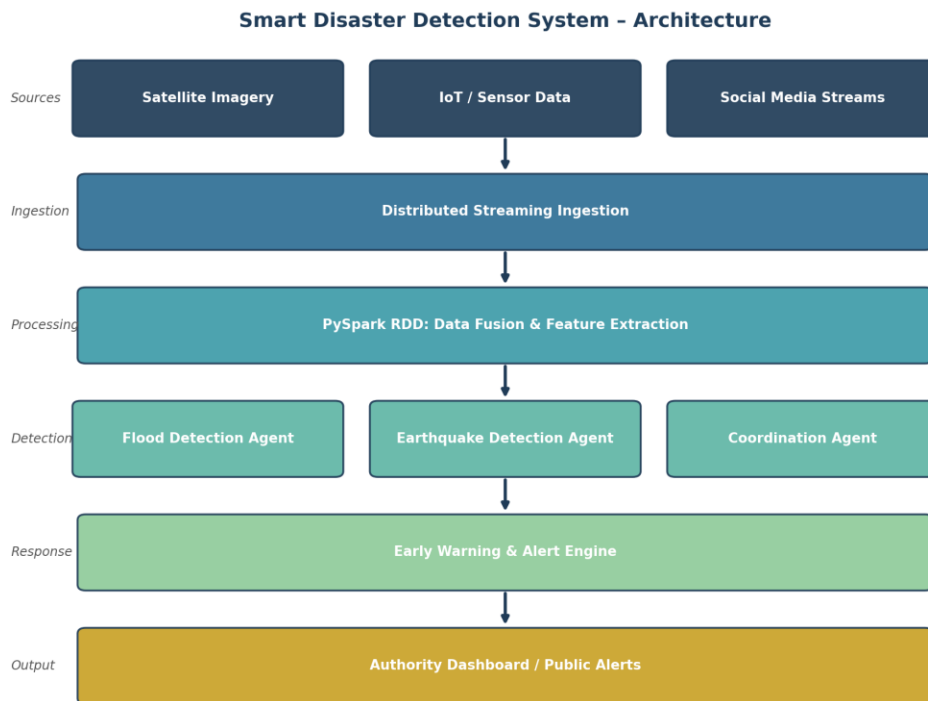
Question 2: Smart Disaster Detection System.

1. Problem Understanding & Requirement Analysis

Natural disasters such as floods and earthquakes must be detected as early as possible from heterogeneous, high-volume, and continuously arriving data — satellite imagery, ground and water-level sensor readings, and social-media posts. The core requirements are to fuse structurally different data types (image, numeric sensor, and unstructured text) into a common representation, to process this data at a scale and speed that supports early warning rather than after-the-fact analysis, and to coordinate the resulting alerts into an actionable response strategy. The system must tolerate noisy and incomplete sensor feeds, avoid false alarms caused by unrelated social-media chatter, and remain operational even if part of the infrastructure fails during the disaster itself.

2. Proposed System Architecture

The architecture begins with a multi-source layer collecting satellite imagery, sensor telemetry, and social-media streams. A distributed streaming-ingestion layer normalises these heterogeneous feeds into a common event format. The processing layer applies PySpark RDD operations for data fusion and feature extraction across all three sources. A detection layer of intelligent agents — a flood-detection agent, an earthquake-detection agent, and a coordination agent — evaluates the fused features against learned thresholds and historical patterns. Confirmed detections flow into an early-warning and alert engine, which finally publishes alerts to an authority dashboard and public-facing notification channels.



3. PySpark & RDD-Based Distributed Processing Design

Satellite images are pre-processed into feature vectors (such as changes in water coverage) and represented as key-value RDDs keyed by geographic grid cell; sensor readings are parsed with map into (location, timestamp, reading) tuples; and social-media posts are filtered and tokenised, keeping only geotagged, disaster-related content. A join operation combines the three RDDs on a common spatial-temporal key (grid cell and time window), producing a fused feature RDD for each region. reduceByKey aggregates readings within each grid cell, and mapPartitions performs the more expensive per-partition anomaly scoring so that model inference is batched rather than invoked per record. Broadcast variables distribute the static grid-to-region mapping and pretrained detection thresholds to all executors, avoiding repeated network transfer. Because rapid response is critical, intermediate fused RDDs are cached, and Spark's DAG scheduler is relied upon to recompute only the affected lineage if a worker node fails during the disaster window.

4. Intelligent Agent Integration

The Flood Detection Agent specialises in interpreting water-level sensor trends and satellite-derived surface-water changes; the Earthquake Detection Agent specialises in seismic-sensor amplitude and frequency patterns; both operate independently on the fused RDD output for their respective feature sets. The Coordination Agent aggregates the outputs of both detection agents together with the social-media

corroboration signal, resolves conflicting or partial evidence, decides whether the combined evidence crosses the alert threshold, and selects an appropriate response strategy (e.g., regional evacuation notice versus localised advisory). This layered agent design keeps each agent focused on a well-defined sub-problem while the coordination agent handles the cross-source decision-making.

5. Scalability & Distributed Computing Design

Grid-cell-based partitioning of RDDs distributes the geographic processing load evenly across the cluster and allows regions to be processed independently and in parallel. RDD lineage provides fault tolerance so that node failures during a live event do not halt processing of unaffected regions. Structured streaming with short micro-batch intervals balances near-real-time responsiveness against throughput, and the cluster can be scaled elastically ahead of forecast severe-weather events to provision additional capacity proactively.

6. Data Flow & Processing Pipeline

Multi-source raw data → streaming ingestion and normalisation → parallel feature-extraction RDDs per source → spatial-temporal join and fusion → per-hazard agent scoring → coordination-agent decision → alert generation → dashboard and public notification. This linear-with-branches flow keeps each processing stage traceable and independently testable.

7. Innovation & Novel Aspects

The system's innovation is in fusing structurally different data modalities (imagery, sensor, and social text) at the grid-cell level before detection, and in using a coordination agent that resolves cross-source evidence rather than relying on any single data source — reducing both missed detections and false alarms compared with single-source early-warning systems.

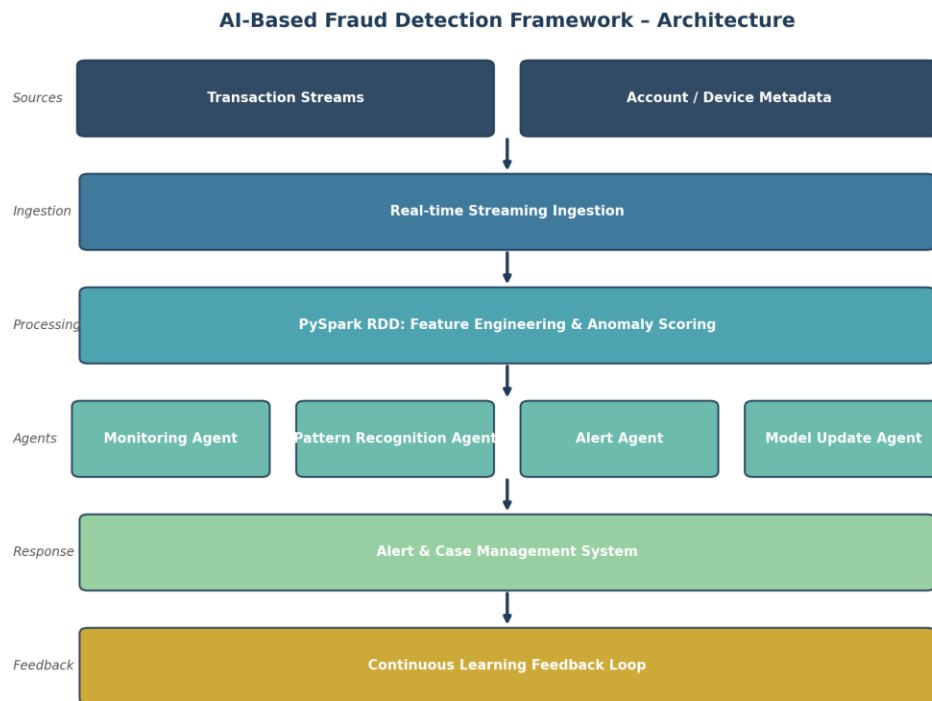
Question 3: AI-Based Fraud Detection Framework.

1. Problem Understanding & Requirement Analysis

Financial institutions process millions of transactions per day, and fraudulent transactions represent a very small fraction of the overall volume, making this a highly imbalanced, high-throughput, low-latency problem. The system must score every transaction against normal behaviour patterns within milliseconds of the transaction occurring, minimise false positives (which frustrate genuine customers), minimise false negatives (which allow fraud through), and continuously adapt to new fraud patterns as fraudsters change their tactics. It must also process at the scale of a national or global payment network, and be auditable so that flagged transactions can be reviewed by a human investigator.

2. Proposed System Architecture

Transaction and account/device metadata streams enter through a real-time streaming-ingestion layer. The processing layer performs PySpark RDD-based feature engineering and anomaly scoring on each transaction, in the context of the account's recent history. A multi-agent layer — comprising a monitoring agent, a pattern-recognition agent, an alert agent, and a model-update agent — collaboratively evaluates each transaction. Suspicious transactions are routed to an alert-and-case-management system for human review, and outcomes from that review feed back into a continuous-learning loop that refines the anomaly-detection model over time.



3. PySpark & RDD-Based Distributed Processing Design

Each incoming transaction is transformed with `map` into a feature vector combining transaction attributes (amount, merchant category, location) with account-level aggregates. `groupByKey` and `reduceByKey` compute rolling per-account statistics such as average transaction amount and transaction frequency over sliding windows, which serve as the baseline against which new transactions are compared. The anomaly-scoring step — for example, a distance-based or isolation-forest-style score — is applied with `mapPartitions` so that the model is loaded once per partition rather than once per record, significantly reducing overhead. `join` combines the transaction RDD with a broadcast RDD of known device and merchant risk scores. Persisted (cached) account-history RDDs avoid repeated recomputation of rolling statistics for high-frequency accounts, and accumulators track the running count of flagged transactions for operational monitoring dashboards.

4. Intelligent Agent Integration

The Monitoring Agent continuously watches the stream of RDD-computed anomaly scores for threshold breaches. The Pattern Recognition Agent looks for coordinated or sequential suspicious behaviour across multiple transactions or accounts (such as a rapid sequence of small transactions preceding a large one). The Alert Agent consolidates the outputs of the previous two agents, ranks alerts by severity, and routes them to human investigators through the case-management system. The Model Update Agent ingests

investigator decisions (confirmed fraud versus false positive) and periodically triggers retraining or threshold recalibration, closing the feedback loop so detection accuracy improves continuously rather than remaining static.

5. Scalability & Distributed Computing Design

Transactions are partitioned by account-id hash so that per-account rolling computations stay within a single partition wherever possible, minimising shuffle. RDD lineage-based fault tolerance ensures that a node failure does not silently drop transactions, and Spark Structured Streaming's micro-batch model provides a practical balance between the sub-second latency financial institutions require and the throughput needed to process national-scale transaction volumes. Elastic executor scaling accommodates predictable peaks such as holiday shopping periods.

6. Data Flow & Processing Pipeline

Transaction stream → real-time ingestion → feature-engineering RDDs → rolling account-history aggregation → anomaly-scoring RDD → multi-agent evaluation → alert generation and case management → investigator feedback → model-update loop back into the anomaly-scoring stage. This closed-loop flow is what allows the framework to keep improving after deployment.

7. Innovation & Novel Aspects

The framework's innovation lies in the closed continuous-learning loop driven by the Model Update Agent, which turns every human investigation outcome into training signal automatically, and in the Pattern Recognition Agent's cross-transaction sequence analysis, which catches coordinated fraud patterns that single-transaction scoring would miss.

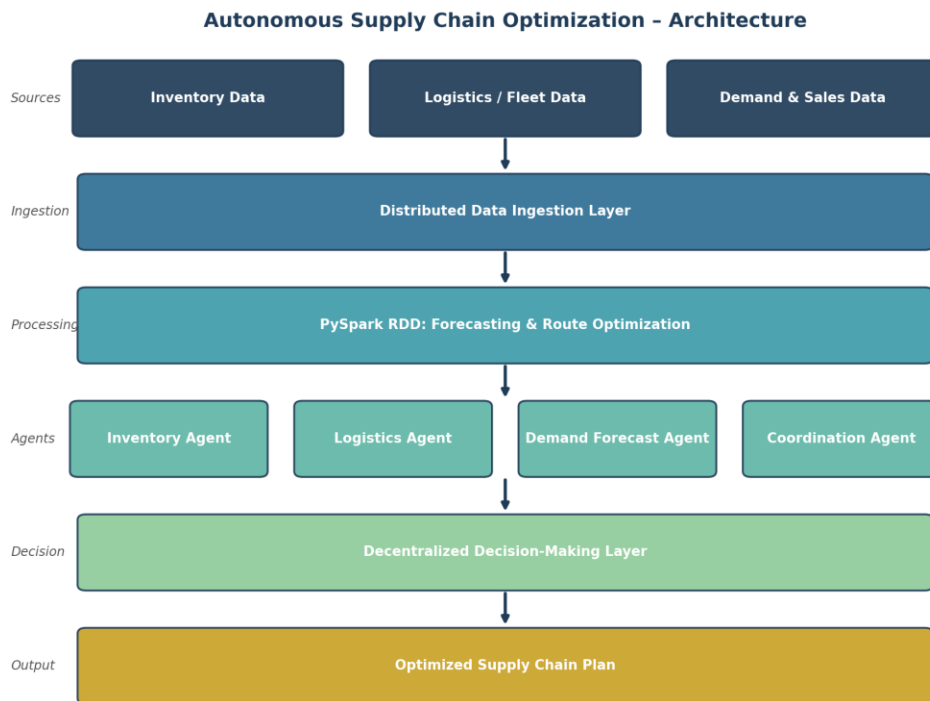
Question 4: Autonomous Supply Chain Optimization System

1. Problem Understanding & Requirement Analysis

A modern supply chain generates continuous data across inventory levels, logistics and fleet movements, and demand signals across many warehouses and retail locations. The design goal is a system that forecasts demand accurately, optimises inventory and delivery routes at scale, and allows decentralised decision-making so that individual warehouses or regions can react quickly to local conditions without waiting for a single centralised planner. Requirements include handling large, multi-source datasets, supporting near-real-time re-optimisation as conditions change (e.g., a delayed shipment or a demand spike), and minimising total cost while maintaining service-level targets.

2. Proposed System Architecture

Inventory data, logistics/fleet data, and demand/sales data are collected through a distributed data-ingestion layer. The processing layer applies PySpark RDD transformations for demand forecasting and route optimisation. A multi-agent layer — an inventory agent, a logistics agent, a demand-forecast agent, and a coordination agent — interacts to translate the processed data into decentralised operational decisions. These decisions are consolidated in a decentralised decision-making layer, which produces the final optimised supply-chain plan (replenishment orders, delivery routes, and stock allocations).



3. PySpark & RDD-Based Distributed Processing Design

Inventory, logistics, and sales records are ingested and parsed with `map` into structured RDDs keyed by warehouse or region. `flatMap` expands order-line-item records into per-SKU transactions for granular analysis. `reduceByKey` aggregates regional demand and stock-level statistics, forming the input to the forecasting agent's time-series models, which are applied per-region using `mapPartitions` to parallelise independent regional forecasts across the cluster. For route optimisation, a graph of delivery legs is represented as an RDD of (origin, destination, cost) edges, and iterative shortest-path-style transformations (similar to Spark's GraphX Pregel-style iteration) are applied to compute cost-minimising routes. Broadcast variables distribute static data such as the warehouse network topology and vehicle-capacity constraints to every executor, and persisted RDDs cache intermediate demand forecasts that multiple downstream agents reuse without recomputation.

4. Intelligent Agent Integration

The Inventory Agent monitors stock levels per warehouse and issues replenishment recommendations based on the demand-forecast agent's output. The Logistics Agent selects delivery routes and vehicle assignments using the optimised route RDDs. The Demand Forecast Agent supplies short-term and medium-term demand predictions per region, feeding both the inventory and logistics agents. The Coordination Agent resolves conflicts between agents — for example, when the inventory agent's

requested replenishment quantity exceeds available logistics capacity — enabling decentralised decisions to be made locally by each specialised agent while remaining globally consistent through periodic coordination rather than a single centralised optimiser.

5. Scalability & Distributed Computing Design

Partitioning RDDs by warehouse/region allows regional forecasting and routing computations to proceed fully in parallel with minimal cross-partition communication. RDD lineage-based recovery ensures that a failure in one region's processing does not affect others. The decentralised agent design itself contributes to scalability, since each region's agents can make local decisions without waiting on a global lock, and the coordination agent only intervenes for conflicts or exceptions, reducing the volume of cross-cluster synchronisation required.

6. Data Flow & Processing Pipeline

Inventory/logistics/demand data → distributed ingestion → per-region feature and demand RDDs → forecasting and route-optimisation RDD transformations → agent-level decentralised decisions (inventory, logistics, demand) → coordination-agent conflict resolution → consolidated optimised supply-chain plan. Each region's data flow can execute independently, joining only at the coordination step.

7. Innovation & Novel Aspects

The innovation is the shift from a single centralised optimiser to genuinely decentralised, region-local agent decisions that are only reconciled when necessary by the coordination agent — reducing decision latency and making the system resilient to the failure or delay of any single region's plan.

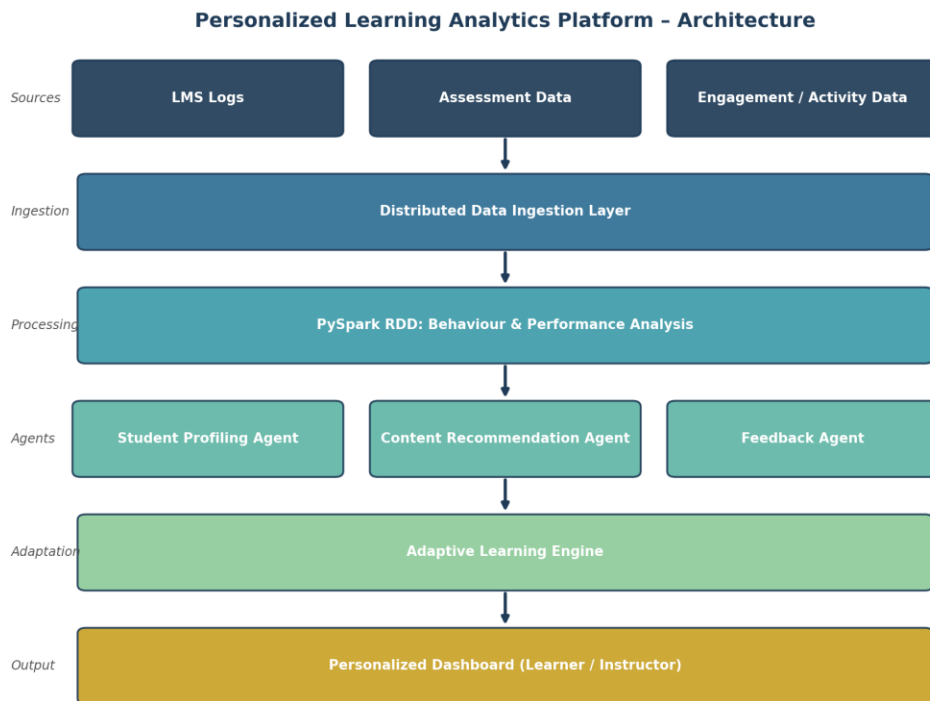
Question 5: Personalized Learning Analytics Platform.

1. Problem Understanding & Requirement Analysis

An adaptive learning platform must analyse learning-management-system (LMS) logs, assessment results, and engagement/activity data for thousands of concurrent learners, and use that analysis to personalise content delivery for each learner individually. Requirements include processing high-volume behavioural data at scale, identifying each learner's strengths, weaknesses, and pace, delivering real-time or near-real-time feedback and content adjustments, and doing so without requiring manual intervention from instructors for every learner. The system must also remain useful to instructors by surfacing cohort-level insights alongside individual personalisation.

2. Proposed System Architecture

LMS logs, assessment data, and engagement/activity data are collected through a distributed data-ingestion layer. PySpark RDD processing analyses learner behaviour and performance in the processing layer. An intelligent-agent layer — a student-profiling agent, a content-recommendation agent, and a feedback agent — interprets this analysis to drive personalisation decisions. These decisions are applied by an adaptive-learning engine, which finally delivers a personalised dashboard and content stream to each learner as well as aggregated insights to instructors.



3. PySpark & RDD-Based Distributed Processing Design

Raw LMS interaction logs are parsed with `map` into (student-id, activity-type, timestamp, score) tuples, and `filter` removes incomplete or test-account records. `groupByKey` and `reduceByKey` aggregate per-student metrics such as time-on-task, quiz accuracy, and topic-level mastery scores from potentially millions of daily events. `join` combines the behavioural RDD with the assessment-result RDD on student-id to build a unified per-student performance profile. `mapPartitions` is used to apply the mastery-scoring or clustering model once per partition, batching learners together for efficient inference rather than scoring one learner at a time. Broadcast variables distribute the curriculum's topic-dependency graph to every executor so that prerequisite-aware recommendations can be computed locally without repeated lookups, and persisted RDDs cache each student's evolving profile so it can be reused across the profiling, recommendation, and feedback agents within the same processing cycle.

4. Intelligent Agent Integration

The Student Profiling Agent maintains an up-to-date model of each learner's mastery level per topic, learning pace, and engagement pattern, derived from the RDD-computed aggregates. The Content Recommendation Agent uses this profile together with the curriculum dependency graph to select the next best learning resource or practice item for each learner, adjusting difficulty adaptively. The Feedback Agent delivers immediate, targeted feedback after each assessment or activity — for example, flagging a

misunderstood prerequisite concept — and signals the profiling agent to update the learner's model in near real time so that the very next recommendation reflects the latest performance.

5. Scalability & Distributed Computing Design

Partitioning RDDs by student-id allows the platform to process thousands of independent learner profiles fully in parallel, with each partition's computation unaffected by others. RDD lineage-based fault tolerance ensures that no learner's progress data is lost if a worker node fails mid-computation. Micro-batch structured streaming supports near-real-time feedback delivery without requiring a fully event-driven architecture, and the cluster can scale elastically around predictable peaks such as assignment deadlines or examination periods when engagement data volume spikes sharply.

6. Data Flow & Processing Pipeline

LMS/assessment/engagement data → distributed ingestion → per-student behavioural and performance RDDs → profile aggregation and join → agent-driven profiling, recommendation, and feedback → adaptive-learning-engine content selection → personalised learner dashboard and instructor analytics. This flow repeats continuously as new learner activity arrives, keeping each learner's experience current.

7. Innovation & Novel Aspects

The platform's innovation is the tight real-time loop between the Feedback Agent and the Student Profiling Agent, which allows content personalisation to react within the same session rather than only after a batch analysis cycle, combined with prerequisite-aware recommendation driven by a broadcast curriculum-dependency graph.